REAL PROJECT · CLIENT DETAILS REMOVED

Resolving a OneDrive Sync Conflict Cascade

A workstation's OneDrive kept crashing on a "path too long" error. I traced it to a runaway sync conflict and renamed about 130,000 conflict copies without deleting a single record.

ROLE	ENVIRONMENT	TOOLS	TIMEFRAME
Independent IT support	OneDrive for Business, Windows	PowerShell, AI assistant	One working session

~130k
conflict copies renamed

0
records deleted

34%
of tested copies differed from the original

9
dry-run iterations before the first live run

THE SITUATION

A small professional-services firm keeps its records in a shared OneDrive library. On one workstation, OneDrive showed a "path too long" error and closed every time it was dismissed, so the library stopped syncing.

MY ROLE

I diagnosed the root cause and ran the fix on live records. An AI assistant wrote the scripts and explained the mechanics. I ran every step, judged the results and made every risk decision.

INSIDE

Diagnosis: how I found the real cause

Tools and built-in safety rules

Execution: 9 iterations, and what each taught

Risk decisions

Outcome, my role and lessons

WHAT IT SHOWS

- Root-cause analysis: read the evidence instead of treating the symptom
- Safe change on live records: dry runs, counts and spot checks before any execution
- Risk judgement: paused a mass delete when the numbers didn't add up
- Effective use of AI: fed back exact errors each round so fixes came from evidence

Shared with the firm's permission. The firm, its people and identifying details are removed.

The problem

One workstation kept showing "We can't sync this item because the path is too long." Dismissing it closed OneDrive entirely. A second workstation on the same library showed no errors. The offending filename ended with the same computer name repeated five times.

Diagnosis

The cause was a sync conflict loop during one overnight event, not deep folders. Three pieces of evidence pointed to it:

1. **The repeated suffix was this workstation's own name.** OneDrive keeps both versions in a conflict and adds the computer name to the copy. Five repeats meant the conflict had been "resolved" over and over.
2. **Every copy shared one timestamp and one file size.** Each document appeared up to six times, all modified within the same minute. That ruled out genuine older versions and pointed to a single bulk event.
3. **Only this workstation was affected.** The longest copies were too long to upload, so they existed only locally. The other workstation mirrors the cloud and never received them.

The most likely trigger was OneDrive being reset or relinked while keeping its local files, so every local file was treated as a conflict. That remains unconfirmed.

Tools and safety rules

The core tool was one PowerShell script with several modes, each run as a dry run first and then with an `-Execute` switch.

Mode or script	What it does	Downloads files?
Compare (default)	Hashes each copy against its original with SHA-256 and moves exact matches to a quarantine folder outside OneDrive	Yes
`-RenameDifferent`	Renames copies whose content differs to a short `Name (conflict N)`, kept beside the original	Yes
`-ShortenOnly`	Renames every suffixed file and folder to a short name without reading it, so it works with OneDrive closed	No
`-DeleteCopies`	Deletes conflict copies but never the last copy of a document	No
Keep-OneDriveRunning	Restarts OneDrive if it closes during a long run	No
Measure-ConflictSize	Reports the total size of conflict copies and free disk space before any download	No

> **Safety rules in every mode:** dry run by default · only byte-identical copies are ever moved · quarantine, never delete · no mode can remove every version of a document · a CSV audit log of every old and new path · downloaded files returned to online-only right after hashing.

Execution: 9 iterations

The first working run took nine iterations over about three hours. Every failure happened in dry-run mode, so none touched a file.

#	What went wrong	Cause	Fix
1	Script ran against a placeholder path	Instructions still had placeholder text	Fail-fast check for a missing folder
2	Null path error; output sent to the printer	A helper named `LP` collided with the built-in `lp` alias	Helpers renamed to unique verb-noun names
3	All 69 originals unreadable	PowerShell 5.1's hashing cmdlet mishandles long-path syntax	Switched to direct .NET file calls, with real errors logged
4	33 files unreadable	"Cloud file provider is not running": OneDrive had closed	Diagnosed as OneDrive failing, not the script
5	Clean run: 193 identical, 101 different	Working as designed	Added an option to rename differing copies
6	OneDrive kept closing mid-run	The long-path error dialog blocked it	Auto-restart watcher plus retry logic
7	Watcher never restarted OneDrive	The dialog kept the process alive, waiting for a click	**Rename-only mode that needs no OneDrive at all**
8	Risk of filling local storage	Comparisons download cloud-only files	Files returned to online-only after hashing; a size check script
9	Same problem in another folder	The event affected the whole library	Rename-only mode run on the entire library

The turning point was iteration 7: instead of keeping OneDrive alive long enough to compare files, remove the long names first with OneDrive closed, then compare later.

Risk decisions

The key decision was to stop at renaming and defer all deletion. In the first test folder, 101 of 294 copies differed from their original byte for byte, so a blanket delete could have destroyed a unique signed or amended version of a document.

- **Verified before acting at scale.** A dry run of the delete mode reported about 130,000 deletions, 13 times the earlier estimate. I paused, saved the log, and spot-checked 15 random files and a per-folder breakdown.
- **Tiered risk by folder type.** Archives carried low risk. Active client folders (about 13,000 copies, including open files) carried high risk and were set aside for compare mode only.
- **Chose reversibility.** Renaming stopped the crashes completely while keeping every byte, so deletion could wait for a sampled review.
- **Escalated what wasn't mine to fix.** A spot check surfaced an unrelated legacy issue in an archive folder. I documented it with a read-only scan and passed it to the owner instead of cleaning it up.

Outcome

Measure	Result
Conflict copies found across the library	About 130,000
Records deleted	0 (renamed only, by choice)
Copies proven identical in the first test folder	193 of 294 (66%)

Measure	Result
Copies differing from the original in that folder	101 of 294 (34%)
Copies in active client folders, held for review	About 13,000
Time to stable sync	One working session

OneDrive stopped crashing after the rename-only pass. CSV audit logs map every original filename to its new one.

My role

I owned the problem, the environment and every decision. The AI assistant wrote the scripts and explained the mechanics. My work was to recognise the evidence, run each step safely on live data, report results accurately, and decide what risk was acceptable.

Skill	Where it showed
Root-cause analysis	Read the repeated computer name and identical timestamps as a conflict loop, not a folder-depth problem
Safe change on live records	Ran every step as a dry run, checked counts and samples, then executed
Risk judgement	Paused a mass delete, tiered folders by risk, and chose reversible renaming
Records management	Kept audit logs of every filename change and protected active client files
Escalation	Routed an unrelated finding to the owner instead of acting on it
Effective use of AI tools	Fed back exact error output each iteration so problems were fixed from evidence, not guesses
Technical execution	Ran and troubleshot PowerShell on Windows, including long paths and OneDrive Files On-Demand

Lessons learned

- **Fix the crash first; decide about deletion later, with evidence.**
- **Naming discipline prevents this.** Saving files under an email subject line used up most of the path length. A convention like `YYYY-MM-DD Document type - Short description.pdf`, with the client carried by the folder, leaves headroom.
- **Duplicate folder trees multiply every problem.** The same tree existed in two places, so every issue appeared twice.
- **Never reset or relink a sync client with unsynced local files.** That is the most likely trigger of the original event.
- **Sync only what a device needs.** Unticking archive folders reduces both scan time and exposure.